Introduction to Rails

Topics Covered: Scaffolding, database migrations, routing, forms

In this worksheet we will introduce you to the Ruby on Rails web framework by building a simple ecommerce application. You can use existing sites like Amazon as inspiration.

If you have difficulties please refer to the content on https://vle.shef.ac.uk, the Rails guides at https://guides.rubyonrails.org/v8.0/, or ask for help.

Preparation

Check out the starter code with:

```
git clone git@git.shefcompsci.org.uk:com213-2025-
26/materials/introduction-to-rails.git
```

Then run the following commands to setup the project:

```
cd introduction-to-rails
bin/setup
```

You can then run

```
bundle exec rails s
```

and in a second terminal or tab

```
bin/shakapacker-dev-server
```

to start up the application. You can now visit http://localhost:3000 to see your application running.

Task 1 - Managing Products

The first requirement of our ecommerce application is to be able to manage **products** that a potential customer can buy. To do this we need to be able to add, edit and delete products. This is a common set of tasks and the Rails framework includes a **scaffold** generator that, when run, provides all of the code needed (database migration, model, controller and view files).

We need to store some information about products. For now, we will only store a `name`, `description` and `cost`. To do this, open a terminal and change into your newly created application's directory, then run the

following command:

```
bundle exec rails g scaffold Product name:string description:text
cost:decimal
```

The meaning of the parts of this scaffold command is as follows:

- `bundle exec` – Use a gem (Rails) bundled with our app to execute a command.
- `rails g scaffold` – g is short for generate, so Rails will generate a new scaffold.
- `<class i.e. Product> <list of database columns and types>` - create a model class and database migration for the class name, followed by a list of database columns that should be stored when the table is created with their types e.g. string, integer, text, date, decimal, etc.

One of the files this scaffold generator creates is a database migration. Open the file `db/migrate/<timestamp created>_create_products.rb` in an editor. It should contain:

```
class CreateProducts < ActiveRecord::Migration[8.0]
  def change
    create_table :products do |t|
      t.string :name
      t.text :description
      t.decimal :cost

      t.timestamps
    end
  end
end
```

It is important to note that your database **WILL NOT** be automatically updated when you create a scaffold. You need to execute the following command in the terminal:

```
bundle exec rails db:migrate
```

This command looks for any existing migration files that have not already been run and updates the database structure accordingly. Consequently, we can commit these files into version control and team members can run the command above to make sure everyone is developing using the same version of the database.

Open the `config/routes.rb` file, where you will see the scaffold generator added this:

```
resources :products
```

This line of code will add all of the relevant paths to pages that will allow us to add, edit, delete and show products. Running the following command in the terminal will show you all of the paths in your application:

```
bundle exec rails routes
```

The output should start with:

```
      Prefix   Verb   URI Pattern                  Controller#Action
      products GET    /products(.:format)          products#index
               POST   /products(.:format)          products#create
   new_product GET    /products/new(.:format)      products#new
  edit_product GET    /products/:id/edit(.:format) products#edit
       product GET    /products/:id(.:format)      products#show
               PATCH  /products/:id(.:format)      products#update
               PUT    /products/:id(.:format)      products#update
               DELETE /products/:id(.:format)      products#destroy
```

These are all of the paths that the `resources :products` code actually creates:

- The first column signifies the path helper that you can use in Rails to access the particular page (i.e. to create a link).
- The second column shows the HTTP verb this path will use, i.e. GET is used when you want to access the content of a page when clicking a link.
- The third column shows the actual URL that will be displayed in your browser when accessing the page.
- The fourth column shows the Rails controller name and controller method that will be executed when this path is accessed. The format of this will always be `<controller name>#<controller method>`.

We want to make the products index page the one we see when we access the `root` page of our application (i.e. when we go to our home page). Currently, we would need to go to `/products` in a browser to do this.

Open the `config/routes.rb` file and change the following code:

```
root "pages#home"
```

to

```
root "products#index"
```

This code says that when we go to the `/` path in our application the `index` method of the `app/controllers/products_controller.rb` controller will be run and the the template in `app/views/products/index.html.haml` will be used to render output to the browser.

Start your application (run `bundle exec rails s`) and then go to http://localhost:3000 in a web browser to see what you have done

You should now be able to add, edit, delete and show products in your ecommerce application. Spend some time interacting with your application to see what the scaffold generates for you. It is also important to try and understand how this works in terms of the code in the `app/controllers/products_controller.rb` file and the view files in the `app/views/products` folder.

We want to add some additional information to the top of our home page to show the customer how many products are available to buy without looking at the table. To do this, open the `app/views/products/index.html.haml` file. Add the following code in bold between the existing header and the "New Product" link:

```
.card-header.d-flex.align-items-center
  %span Listing Products
  %span (There are #{@products.size} products available.)
  = link_to 'New Product', new_product_path, class: 'btn btn-outline-
secondary ms-auto'
```

Refresh the home page in your browser and you should see the extra text next to the "Listing products" heading.

If you open the `app/controllers/products_controller.rb` file you will see the following code in the index action:

```
# GET /products
def index
  @products = Product.all
end
```

The `Product.all` code retrieves all of the products added to the database, puts them in an array and assigns it to an `@products` instance variable. This instance variable in the index controller method can then be accessed in the related view (i.e. `app/views/products/index.html.haml`) which is why we are able to add a span showing the size of the `@products` array. The `#{}` syntax allows us to execute Ruby code within a view and output the result to the page.

Lets take a look at the update action that has been generated in the `ProductsController`. The first line starts with `if @product.update(product_params)`. This is immediately referencing the instance variable `@product` which has not been defined in the update action. It was defined in the method `set_product` which you can find towards the bottom of the controller.

```
# Use callbacks to share common setup or constraints between actions.
def set_product
  @product = Product.find(params.expect(:id))
end
```

This method loads the `@product` instance from the id posted to the action. This method is run as a callback which you can see at the top of the controller with `before_action :set_product, only: %i[ show edit update destroy ]`. This line tells Rails to call this method before the show, edit, update and destroy actions are run. As a result the `@product` instance variable is available in those actions.

You will notice that after you `create` a product, it takes you to the show page for that new product. We want it to go back to the index page instead.

If you open the `app/controllers/products_controller.rb` file you will see the following code in the `create` action:

```
# POST /products
def create
  @product = Product.new(product_params)

  if @product.save
    redirect_to @product, notice: "Product was successfully created."
  else
    render :new, status: :unprocessable_content
  end
end
```

Look at the code in bold. When a product is saved and added to the database based on the fields you fill in on the form, it redirects to `@product` (which in Rails means go to the show action for this new product). Change it to the following:

```
# POST /products
def create
  @product = Product.new(product_params)

  if @product.save
    redirect_to products_path, notice: "Product was successfully created."
  else
    render :new, status: :unprocessable_content
  end
end
```

You will see that we are now redirecting to the `products_path` which means go back to the index page instead. If you run:

```
bundle exec rails routes
```

in your terminal, the first column shows the path prefixes Rails uses. If we append `_path` onto the end of the prefix this will be the name of the Rails helper method for that path. e.g. a prefix of `edit_product` will

have a helper method of `edit_product_path` which will return the relative path to that route.

Create a new product in your browser. It should now redirect back to the index page.

Change the `update` action to redirect back to the `index` action when the product has been successfully updated. Test this in your browser.

Task 2 - Categories

You have now used Rails to generate a scaffold, and have seen how quickly it can be used to create a simple feature. In this task we will do the same for categories, but without using scaffolds. It is unlikely that a scaffold will always provide the best solution without modification, so you should also become familiar with the manual process.

1. Generate a model and a migration to make the database table for categories:

```
bundle exec rails g model Category name:string code:string
```

This functions very similarly to the scaffold generator used earlier, but creates fewer files.

The files that are created are as follows:

- `db/migrate/<timestamp>_create_categories.rb` – A migration file to create the `categories` table, with a `name` and `code` column.
- `app/models/category.rb` – A category model.
- `spec/models/category_spec.rb` – A test file. We will look into these in more detail in a future worksheet.
- `spec/factories/categories.rb` – A category factory. This will allow us to quickly make categories for testing purposes. We will look into these in more detail in a future worksheet.

2. Migrate your database to create the new table:

```
bundle exec rails db:migrate
```

3. Now we have a model and an updated database, we need to make a controller. Create a new file `app/controllers/categories_controller.rb`. Notice our model file is singular `category.rb` but our controller is plural `categories_controller.rb`. This is a Rails convention that you will need to adhere to in order for your application to work correctly.

Inside your new file create an empty categories controller. This will look like the following:

```
class CategoriesController < ApplicationController

end
```

This file is going to be where the requests for everything category-related are handled. Today we are just going to make the standard actions which allow us to create, read, update and destroy categories. This is exactly the same functionality as we have for `Products`.

4. The next step is being able to create a new category. For this we need to create two actions in our categories controller. `new` to serve the page with the form, and `create` to store a new category in the database.

The code for this is as below:

```
def new
  @category = Category.new
end

def create
  category_params = params.require(:category).permit(:code, :name)
  @category = Category.new(category_params)

  if @category.save
    redirect_to new_category_path, notice: 'Category was created.'
  else
    render :new
  end
end
```

The `new` action simply builds a new category, so that the view has an object to render with.

The `create` action takes the parameters that are posted to it, and allows the `code` and `name` attributes to be assigned to the model. It then attempts to save the new record.

5. Now we have some code to create a new category we need to make it accessible via our application. We need to edit `config/routes.rb` and add a **route** for our new code. A route provides a mapping from a URL to a controller action. In this case we want `/categories/new` to go to our `new` action in our `categories` controller.

In `config/routes.rb` add the following line:

```
resources :categories
```

This will create all of the routes we need to create, read, update and destroy categories. Run this command:

```
bundle exec rails routes
```

to see the new routes that have been created.

6. Now we have a model and a controller, the only thing missing is our view.

Create a new file `app/views/categories/new.html.haml` for the view and add this content:

```
.row.justify-content-center
  .col-sm-8.col-lg-6
    .card
      .card-header New Category
      = simple_form_for(@category) do |f|
        .card-body
          = f.input :code
          = f.input :name
        .card-footer.d-flex
          = f.button :submit, class: 'btn btn-primary'
```

Simple Form is a gem that we use to make creating forms in Rails easier. It uses Active Record models to assume sensible defaults about where the form should post to. In this case, we have given it a category so it knows to go to the `CategoriesController`, and as we have a new record, it knows to go to the `create` action that we made earlier.

7. Make sure you have a Rails server running and visit: http://localhost:3000/categories/new. If you have done everything correctly, you should see a form to create a new category. Try creating a few categories to make sure that it works.

Task 3

You should now have a fully working products scaffold, and the ability to create categories that you have implemented yourself. Create an index page for categories and link to it from the main menu. You can use your existing products scaffold to help you.

Task 4

Implement the ability to update categories. You can use your existing products scaffold to help you.

Task 5

Implement the ability to delete categories. You can use your existing products scaffold to help you.